

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS



**ANÁLISIS DE CUMPLIMIENTO DE METAS DE TRADING EN INTERBANK
UTILIZANDO CLOUD**

Curso: SISTEMA DE INTELIGENCIA DE NEGOCIOS
Sección: SI 807-U

Alumnos:

Gamboa Checnes Joel Fernando	20212635D
Aymachoque Aymachoque Luis Jairo	20191144G
Laureano Hidalgo Jordan Cesar	20212559F

Ciclo: 25-2

Lima, 15 de diciembre de 2025

JUSTIFICACIÓN DEL USO DE LA NUBE

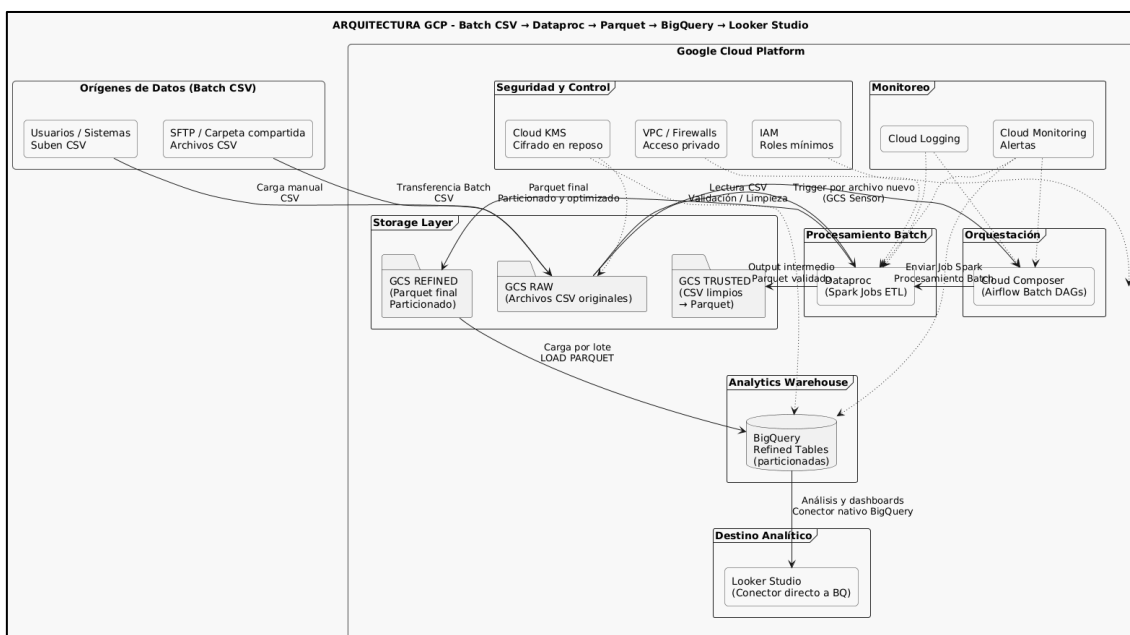
Se eligió Google Cloud Platform (GCP) porque ofrece una base moderna, fácil de usar y alineada con las necesidades del área de Trading. BigQuery permite analizar grandes volúmenes de información con gran velocidad, mientras que Dataproc brinda flexibilidad para procesamientos más complejos, especialmente considerando que la mayoría de nuestros datos se manejan en batch. Además, GCP ofrece un modelo de costos claro y predecible. En conjunto, esta plataforma permite que el equipo de BI responda con rapidez ante la incertidumbre del mercado y genere estrategias oportunas para cumplir las metas de utilidad.

Comparativa

Característica	Google Cloud Platform (GCP) (Elegida)	Amazon Web Services (AWS)	Microsoft Azure
1. Escalabilidad (DW)	Automática y Sin Límites (BigQuery). No nos preocupamos de la infraestructura. El sistema escala solo para manejar cualquier cantidad de datos de Trading. Ventaja Técnica: Máxima agilidad. No hay tiempos muertos esperando que se ajusten los servidores.	Requiere Gestión (Redshift). Tienes que prever el tamaño del servidor. Si sube la carga de trabajo, se debe ajustar manualmente. Nota: Ahora existe también serverless	Requiere Planificación (Synapse). Necesitas definir la capacidad por adelantado.
2. Procesamiento de los CSV (ETL)	Máximo Control (Cloud Dataproc). Permite usar Apache Spark/Hive con total flexibilidad de código para limpiar y transformar los CSV. Ventaja: Ideal cuando se necesita optimizar al máximo el rendimiento de los jobs de transformación pesada (miles de registros).	Máxima Simplicidad (AWS Glue). Es un servicio Serverless que gestiona automáticamente toda la infraestructura de transformación. Ventaja: El equipo se centra solo en la lógica de transformación, sin gestionar servidores. Desventaja: Menor control para optimizaciones manuales de cluster o para usar librerías de Spark muy personalizadas..	Máxima Integración (Data Factory). ADF es el Orquestador Principal que centraliza y gestiona todo el flujo de datos. La transformación pesada se delega a servicios conectados como Azure Synapse o Databricks.
3. Inteligencia Predictiva (ML)	Integración Nativa (Vertex AI). Es la nube mejor integrada para construir modelos de Machine Learning (ML) sobre BigQuery. Ventaja Técnica: Mejores estrategias. Permite crear modelos que predigan si un mes será bueno o malo para la utilidad.	Plataforma Sólida (SageMaker). Excelente, pero requiere más pasos para mover los datos entre el DW (Redshift) y la plataforma de ML.	Buen Ecosistema (ML Studio). Ideal para equipos que usan herramientas de desarrollo de Microsoft.

4. Modelo de Cobro (DW)	Solo por lo que Consultas. BigQuery cobra por el volumen de datos que escaneas al hacer una pregunta (consulta). Ventaja Financiera: Incentiva la eficiencia. Si el equipo diseña bien las tablas, el costo de análisis es muy bajo. Se paga solo por el análisis real, no por un servidor encendido.	Pago por Servidor Encendido. Se paga por hora o minuto que el servidor de Redshift esté activo, sin importar si lo estás usando o no. Riesgo Financiero: Requiere gestión constante para apagarlo.	Pago por Uso de Recursos. Similar a AWS, basado en el tiempo de computación.
5. Previsibilidad de Costos	Muy Alta. BigQuery te dice cuánto costará una consulta antes de ejecutarla. Ventaja Financiera: Cero sorpresas. El equipo puede operar dentro de su presupuesto sin miedo a consultas accidentales y costosas.	Media. El costo está ligado a la complejidad del <i>cluster</i> y su uso, lo que hace más difícil predecir el gasto.	Media. Similar a AWS, el costo es más fácil de predecir con cargas de trabajo constantes.
6. Seguridad de Datos Sensibles	Robusta. Seguridad de nivel global. Ventaja Técnica/Financiera: La seguridad es de las más altas del mercado, esencial para proteger los datos transaccionales y de utilidad del área de Trading.	Líder en Cumplimiento. Muy fuerte en seguridad y cumplimiento normativo.	Excelente para Empresas. Muy fuerte en el sector empresarial, especialmente con Azure AD.

ARQUITECTURA CLOUD DEFINIDA



SELECCIÓN DE SERVICIOS CLOUD Y ARQUITECTURA

1. Orígenes de Datos e Ingesta (Data Sources and Ingestion)

Servicio Utilizado: Google Cloud Storage (GCS)

Orígenes de Datos (CSV): Los datos inician como archivos CSV, subidos por usuarios, sistemas o a través de métodos como SFTP o carpetas compartidas.

Transferencia Batch / Carga Manual: Los archivos CSV son cargados en la capa de almacenamiento.

2. Capa de Almacenamiento (Storage Layer)

GCS actúa como un lago de datos (Data Lake) que almacena los archivos en diferentes etapas del proceso:

- GCS RAW (Archivos CSV originales): Almacena los archivos CSV tal como se reciben.
- GCS TRUSTED (CSV limpios Parquet): Almacena el output intermedio del procesamiento, que incluye archivos CSV que han pasado la validación/limpieza y, posiblemente, su conversión inicial a Parquet.
- GCS REFINED (Parquet final Particionado): Almacena la versión final, particionada y optimizada de los datos, lista para ser cargada en el Data Warehouse.

3. Orquestación y Control del Flujo

Este paso define y gestiona la ejecución del proceso ETL (Extracción, Transformación y Carga):

Cloud Composer (Airflow Batch DAGs): Es el servicio de orquestación central (basado en Apache Airflow). Se utiliza para definir las tuberías de datos (DAGs) y gestionar la secuencia de tareas, como validar la presencia de un archivo y luego iniciar el procesamiento.

4. Procesamiento Batch (Batch Processing)

Esta es la etapa de ETL, donde se transforma el dato crudo en dato estructurado y limpio:

Dataprocc (Spark Jobs ETL): Plataforma para ejecutar clústeres de código abierto (como Apache Spark). Dataprocc lee los datos de GCS RAW/TRUSTED, realiza tareas de Validación / Limpieza, y lleva a cabo la transformación a formato Parquet (un formato columnar optimizado para el análisis).

Relación: Dataprocc lee desde las capas RAW/TRUSTED y escribe el resultado final (Parquet final) en la capa GCS REFINED.

5. Almacén Analítico (Analytics Warehouse)

BigQuery Refined Tables (particionadas): El servicio de Data Warehouse sin servidor de Google.

Relación: Una vez que los datos optimizados están en GCS REFINED, BigQuery utiliza la función LOAD PARQUET (carga por lote) para moverlos eficientemente a tablas particionadas, preparándolos para la consulta y el análisis.

6. Destino Analítico (Analytical Destination)

Looker Studio (Conector directo a BQ): Herramienta de visualización de datos.

Relación: Se conecta directamente a las tablas refinadas de BigQuery para crear Análisis y dashboards, lo que permite a los usuarios finales consumir la información procesada.

Servicios Transversales (Cross-Cutting Services)

Estos servicios aplican a toda la arquitectura para garantizar su seguridad y fiabilidad:

Servicio	Categoría	Función en la Arquitectura
IAM	Seguridad y Control	Gestiona los permisos y el acceso a los servicios de GCP (define <i>Roles mínimos</i>).
Cloud KMS	Seguridad y Control	Proporciona la gestión de claves para el Cifrado en reposo de los datos (por ejemplo, en GCS y BigQuery).
VPC / Firewalls	Seguridad y Control	Configura la red privada y el acceso seguro entre los servicios.
Cloud Logging	Monitoreo	Recopila y almacena logs de todos los servicios para la auditoría y depuración.
Cloud Monitoring Alertas	Monitoreo	Monitorea el estado y el rendimiento del pipeline, y notifica problemas operativos.

En este apartado se mostrarán las configuraciones

IAM:

Rol para el integrante Aymachoque.

Servicios habilitados: Dataproc, Composer, GCS.

	aymachoqueluis26@gmail.com	Administrador de Composer Lector de Dataproc Visualizador de buckets de Storage (Beta)
---	----------------------------	--

Rol integrante Jordan Laureano.

Servicios habilitados: Composer, Dataproc, BigQuery y CGS.

☐	jordan.laureano.h@uni.pe	Administrador de Composer
		Editor de datos de BigQuery
		Lector de Dataproc
		Usuario de trabajo de BigQuery
		Visualizador de buckets de Storage (Beta)
		Visualizador de objetos de Storage

Cifrado KMS.

Llavero de claves multi-regional creada por interfaz:

☐	my-keyring-multi-region	us	(Ninguna)	—	⋮
--------------------------	---	----	-----------	---	---

Creación de Clave:

← Crear clave

Una clave criptográfica es un recurso que se utiliza para cifrar y descifrar datos o para producir y verificar firmas digitales. Puede tener varias combinaciones. [Más información](#)

- Nombre y nivel de protección

Nombre: my-key-multi-region

Nivel de protección: Software
- Material de clave

Key material: Generada
- Propósito y algoritmo

Propósito: Encriptación/desencryptación simétrica

Algoritmo: Clave Simétrica de Google
- Versiones

Período de rotación de claves: 90 días

A partir del: 27/2/26
- Configuración adicional (opcional)

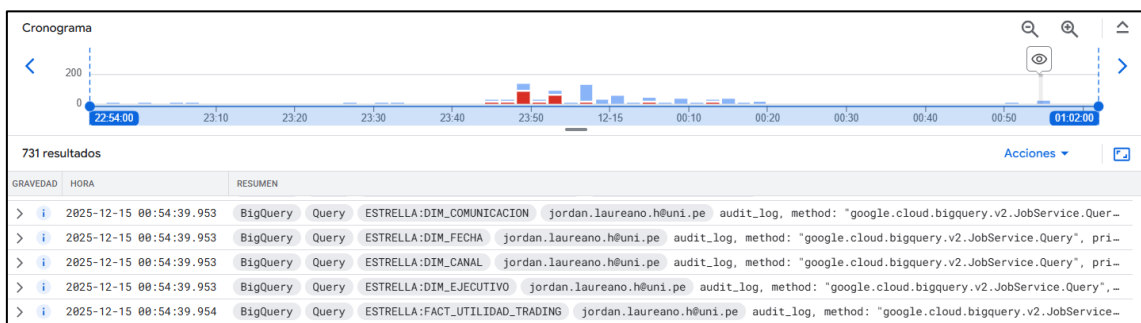
Duración del estado "Programada para destrucción": día(s)

Crear Cancelar

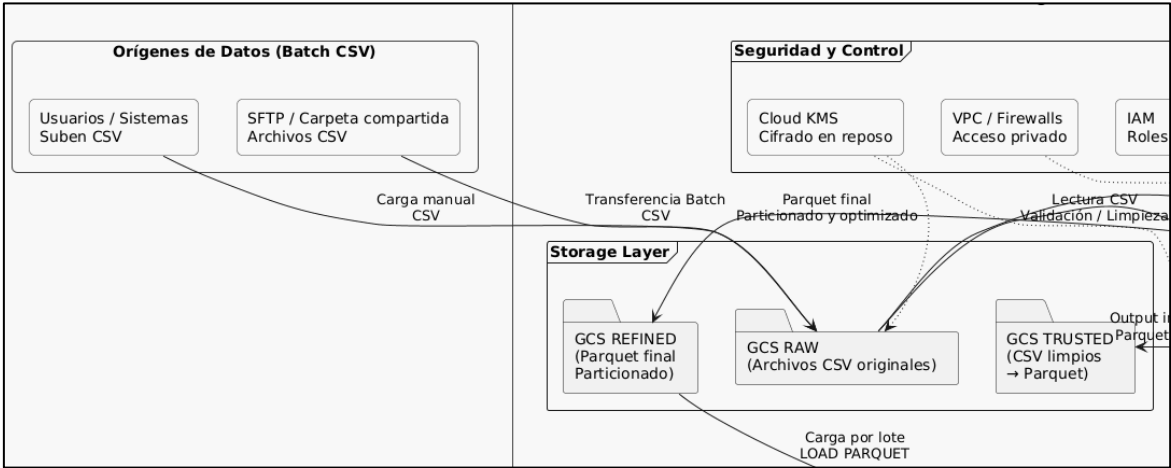
☐	Nombre ↑	Estado ?	Nivel de protección ?	Propósito ?	Próxima rota
☐	my-key-multi-region	✓ Disponible	Software	Encriptación/desencryptación simétrica	27 feb 2026

Es importante que la clave sea multi-region debido a que los buckets que se configuraron como multi-region.

Registros de Logging:



DISEÑO DEL DATALAKE



project-sin-477115-raw	Multi-region	us	Standard
project-sin-477115-refined	Multi-region	us	Standard
project-sin-477115-trusted	Multi-region	us	Standard

Los buckets de Google Cloud Storage (GCS) que están configurados para funcionar como las capas principales de un Data Lake (Lago de Datos) en Google Cloud Platform.

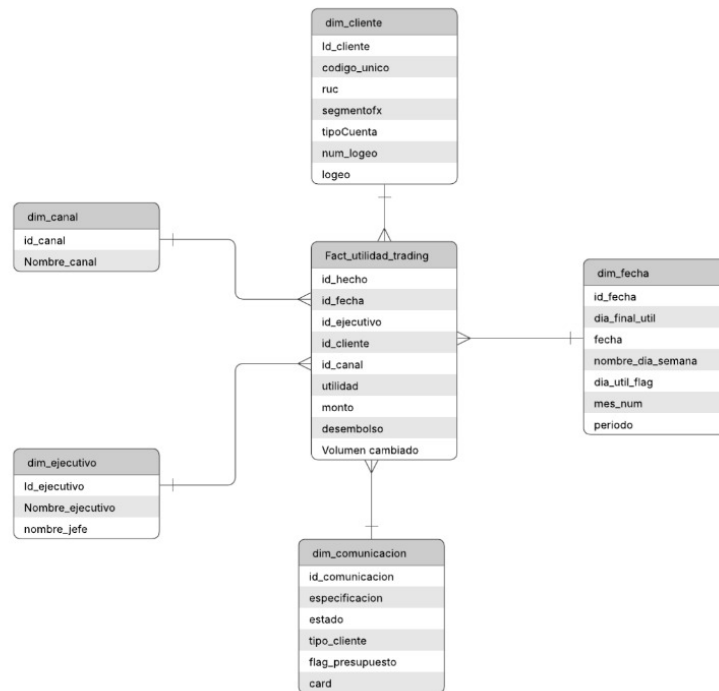
Bucket	Capa del Data Lake	Propósito y Contenido
project-sin-477115-raw	RAW (Cruda)	Almacena los datos en su formato original e inalterado. Aquí se cargan los archivos CSV tal como fueron recibidos de las fuentes de datos.
project-sin-477115-trusted	TRUSTED (Confiable)	Almacena datos que han pasado por una validación y limpieza inicial. Los datos están estandarizados y limpios de errores básicos. Podrían estar convertidos a un formato semi-optimizado como CSV limpio o incluso Parquet.
project-sin-477115-refined	REFINED (Curada)	Almacena la versión de los datos optimizada, transformada y lista para el análisis (Data Marts). Están en formatos de columna como Parquet u ORC y están particionados para una lectura eficiente por BigQuery.

Metadatos y Configuración Comunes

Todos los buckets comparten las siguientes configuraciones, que son típicas para un Data Lake basado en GCS:

- **Multi-region:** Indica que los datos están almacenados y replicados en múltiples regiones geográficas (US). Esto proporciona alta disponibilidad y resistencia a fallos regionales.
- **Standard:** Clase de almacenamiento ideal para datos a los que se accede o modifica con frecuencia (como los datos que están activamente siendo leídos y escritos por el proceso ETL).

MODELO DIMENSIONAL IMPLEMENTADO



Este es un modelo de estrella típico, diseñado para el análisis de la utilidad de trading (Fact_utilidad_trading).

- Tabla de Hechos Central: Fact_utilidad_trading. Contiene las métricas de negocio como utilidad, monto, desembolso y volumen cambiado, junto con las claves primarias de las dimensiones.
- Tablas de Dimensión: Son cinco tablas que proporcionan contexto a las transacciones:
 - dim_cliente (Cliente)
 - dim_fecha (Tiempo)
 - dim_ejecutivo (Ejecutivo/Agente)
 - dim_canal (Canal de Venta/Origen)
 - dim_comunicacion (Detalles de la Comunicación)

El modelo permite analizar métricas de trading clave por Cliente, Fecha, Ejecutivo, Canal y Comunicación.

PROCESO ETL

Propósito:

Realizar la limpieza de los archivos csv que se tienen y realizar la creación de tablas de hechos y dimensiones, y por último realizar los KPIs.

Origen (Extract)

Los archivos CSV se obtienen de los sistemas de información del Área de Trading de la empresa Interbank. Esos archivos se almacenan en crudo en un bucket de Google Cloud Storage.

project-sin-477115-raw																																																							
Ubicación		Clase de almacenamiento	Acceso público	Protección																																																			
us (múltiples regiones en Estados Unidos)		Standard	No público	Borrar de forma no definitiva, Control de versiones de objetos																																																			
Objetos	Configuración	Permisos	Protección	Ciclo de vida	Observabilidad	Informes de inventario Operaciones																																																	
Navegador de carpetas																																																							
Depósitos > project-sin-477115-raw > ingesta > 2025-11-30																																																							
Crear carpeta Subir Transferir los datos Otros servicios Más información																																																							
Filtrar solo por prefijo de nombre Filtro Filtrar objetos y carpetas Mostrar Solo objetos activos																																																							
<table> <tr> <th>☐</th><th>Nombre</th><th>Tamaño</th><th>Tipo</th><th>Fecha de creación</th><th>Clase de almacen</th><th></th></tr> <tr> <td>☐</td><td>cli_cambios.csv</td><td>280 KB</td><td>text/csv</td><td>30 nov 2025, 12:21:57 a.m.</td><td>Standard</td><td></td></tr> <tr> <td>☐</td><td>registro_comunicaciones_new_v2...</td><td>24.3 KB</td><td>text/csv</td><td>30 nov 2025, 12:21:57 a.m.</td><td>Standard</td><td></td></tr> <tr> <td>☐</td><td>segmentos_v2.csv</td><td>1 MB</td><td>text/csv</td><td>30 nov 2025, 12:21:58 a.m.</td><td>Standard</td><td></td></tr> <tr> <td>☐</td><td>tiendas_ranking_propio_updated.c...</td><td>4 KB</td><td>text/csv</td><td>30 nov 2025, 12:21:58 a.m.</td><td>Standard</td><td></td></tr> <tr> <td>☐</td><td>tlv_ranking_propio_updated.csv</td><td>2.7 KB</td><td>text/csv</td><td>30 nov 2025, 12:21:58 a.m.</td><td>Standard</td><td></td></tr> <tr> <td>☐</td><td>virtual_ranking_propio_updated.csv</td><td>1.9 KB</td><td>text/csv</td><td>30 nov 2025, 12:21:58 a.m.</td><td>Standard</td><td></td></tr> </table>							☐	Nombre	Tamaño	Tipo	Fecha de creación	Clase de almacen		☐	cli_cambios.csv	280 KB	text/csv	30 nov 2025, 12:21:57 a.m.	Standard		☐	registro_comunicaciones_new_v2...	24.3 KB	text/csv	30 nov 2025, 12:21:57 a.m.	Standard		☐	segmentos_v2.csv	1 MB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard		☐	tiendas_ranking_propio_updated.c...	4 KB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard		☐	tlv_ranking_propio_updated.csv	2.7 KB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard		☐	virtual_ranking_propio_updated.csv	1.9 KB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard	
☐	Nombre	Tamaño	Tipo	Fecha de creación	Clase de almacen																																																		
☐	cli_cambios.csv	280 KB	text/csv	30 nov 2025, 12:21:57 a.m.	Standard																																																		
☐	registro_comunicaciones_new_v2...	24.3 KB	text/csv	30 nov 2025, 12:21:57 a.m.	Standard																																																		
☐	segmentos_v2.csv	1 MB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard																																																		
☐	tiendas_ranking_propio_updated.c...	4 KB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard																																																		
☐	tlv_ranking_propio_updated.csv	2.7 KB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard																																																		
☐	virtual_ranking_propio_updated.csv	1.9 KB	text/csv	30 nov 2025, 12:21:58 a.m.	Standard																																																		

Transformación:

Se realizó la transformación de los datos en el servicio de Dataproc de GCP, donde se realizó primero la limpieza de los datos, todo esto en el entorno de JupyterLab.

```
project-sin-477115 > etl-cluster-batch
File Edit View Run Kernel Git
Filter files by name
Name Last Modified
Untitled.py... 29 minutes ago
Untitled.py... 12 minutes ago

[1]: from pyspark.sql import SparkSession
from pyspark.sql import functions as f
from pyspark.sql.types import StringType

spark = (
    SparkSession.builder
        .appName("limpieza_csv_kpi")
        .getOrCreate()
)

# ===== AYUDAS DEL PROYECTO =====
raw_bucket = "project-sin-477115-raw"
refined_bucket = "project-sin-477115-refined"
fecha_path = "ingesta/2025-11-30" # carpeta que se ve en tu screenshot

base_path = f"gs://{raw_bucket}/{fecha_path}"
output_clean_base = f"gs://{refined_bucket}/{fecha_path}/clean"
output_kpi_path = f"gs://{refined_bucket}/{fecha_path}/kpi"

# ===== ARCHIVOS =====
files = [
    "cli_cambios": f"{base_path}/cli_cambios.csv",
    "registro_comunicaciones_new_v2": f"{base_path}/registro_comunicaciones_new_v2.csv",
    "segmentos_v2": f"{base_path}/segmentos_v2.csv",
    "tiendas_ranking_propio_updated": f"{base_path}/tiendas_ranking_propio_updated.csv",
    "tlv_ranking_propio_updated": f"{base_path}/tlv_ranking_propio_updated.csv",
    "virtual_ranking_propio_updated": f"{base_path}/virtual_ranking_propio_updated.csv",
]

# ===== FUNCIÓN DE LIMPIEZA + KPI =====
def clean_and_kpi(table_name: str, path: str):
    df = (
        spark.read
            .option("header", "true")
            .option("inferSchema", "true")
            .csv(path)
    )

    original_rows = df.count()
    num_columns = len(df.columns)
    string_cols = [f.name for f in df.schema.fields if isinstance(f.dataType, StringType)]
    num_string_cols = len(string_cols)

    # Métricas antes
    if string_cols:
```


SCRIPTS UTILIZADOS

Los siguientes Scripts fueron utilizados para crear la **tabla de hechos y dimensiones**:

```
-- DIM_CANAL
CREATE OR REPLACE TABLE `project-sin-477115.ESTRELLA.DIM_CANAL` AS
SELECT
  ROW_NUMBER() OVER (ORDER BY canal) AS id_canal,
  canal AS nombre_canal
FROM (
  SELECT DISTINCT canal FROM `project-sin-477115.refined_data.tlv_ranking`
  UNION ALL
  SELECT DISTINCT canal FROM `project-sin-477115.refined_data.virtual_ranking`
  UNION ALL
  SELECT DISTINCT canal FROM `project-sin-477115.refined_data.tiendas_ranking`
);
```

Este script SQL tiene como objetivo crear o reemplazar una tabla de dimensiones llamada DIM_CANAL (Dimensión Canal), dentro del esquema ESTRELLA en el proyecto project-sin-477115 de GCP. Para poblar esta tabla, primero combina (utilizando UNION ALL) todos los valores distintos de la columna canal provenientes de tres tablas de origen diferentes (tlv_ranking, virtual_ranking y tiendas_ranking). Luego, a cada uno de estos canales distintos (que se convierten en la columna nombre_canal en la tabla final), le asigna un identificador único, secuencial y ordenado llamado id_canal mediante la función de ventana ROW_NUMBER() OVER (ORDER BY canal), creando así la dimensión que se utilizará para analizar la información por canal en un modelo de datos.

```
-- DIM_EJECUTIVO
CREATE OR REPLACE TABLE `project-sin-477115.ESTRELLA.DIM_EJECUTIVO` AS
SELECT
  ROW_NUMBER() OVER (ORDER BY ejecutivo, jefe) AS id_ejecutivo,
  ejecutivo AS nombre_ejecutivo,
  jefe AS nombre_jefe
FROM (
  SELECT DISTINCT ejecutivo, jefe FROM `project-sin-477115.refined_data.tlv_ranking`
  UNION DISTINCT
  SELECT DISTINCT ejecutivo, jefe FROM `project-sin-477115.refined_data.virtual_ranking`
  UNION DISTINCT
  SELECT DISTINCT ejecutivo, jefe FROM `project-sin-477115.refined_data.tiendas_ranking`
);
```

Este script tiene la función de crear o reemplazar una tabla de dimensiones llamada DIM_EJECUTIVO (Dimensión Ejecutivo) dentro del mismo esquema ESTRELLA del proyecto. Su objetivo es construir una dimensión que contenga información sobre los ejecutivos y sus respectivos jefes. Para lograrlo, primero combina (usando UNION DISTINCT) los pares únicos de ejecutivo y jefe que se encuentran en las tres tablas de origen (tlv_ranking, virtual_ranking y tiendas_ranking). La combinación UNION DISTINCT asegura que no haya duplicados en la lista de pares ejecutivo-jefe. Finalmente, a cada fila resultante (que proporciona el nombre_ejecutivo y el nombre_jefe), le asigna un identificador único, secuencial y ordenado llamado id_ejecutivo utilizando la función de ventana ROW_NUMBER() sobre el orden del ejecutivo y su jefe.

```
-- DIM_FECHA
CREATE OR REPLACE TABLE `project-sin-477115.ESTRELLA.DIM_FECHA` AS
SELECT
  CAST(REPLACE(fecha_raw, '-', '' ) AS INT64) AS id_fecha,
  DATE(fecha_raw) AS fecha,
  CAST(SUBSTR(REPLACE(fecha_raw, '-', ''), 1, 6) AS INT64) AS periodo,
  1 AS dia_util_flag,
  0 AS dia_final_util,
  CAST(SUBSTR(REPLACE(fecha_raw, '-', ''), 1, 4) AS INT64) AS anio,
  CAST(SUBSTR(REPLACE(fecha_raw, '-', ''), 5, 2) AS INT64) AS mes_num,
  'DESCONOCIDO' AS nombre_dia_semana
FROM (
  SELECT DISTINCT dia AS fecha_raw
  FROM `project-sin-477115.refined_data.registro_comunicaciones`
  UNION DISTINCT
  SELECT DISTINCT SUBSTR(fecha, 1, 10) AS fecha_raw
  FROM `project-sin-477115.refined_data.cli_cambios`
);
```

La tabla DIM_FECHA se construye a partir de todos los valores de fecha distintos encontrados en dos tablas de origen (registro_comunicaciones y cli_cambios). El script transforma las fechas sin formato (fecha_raw) en varios atributos de tiempo, lo cual es esencial en cualquier modelo de datos de estrella para permitir el análisis por día, mes, año, etc.

```
-- DIM_CLIENTE
CREATE OR REPLACE TABLE `project-sin-477115.ESTRELLA.DIM_CLIENTE` AS
WITH temp_cambios_clientes_reciente AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY `Codigo Unico`
      ORDER BY fecha DESC
    ) AS rn
  FROM `project-sin-477115.refined_data.cli_cambios`
)
SELECT
  ROW_NUMBER() OVER (ORDER BY u.`Codigo Unico`) AS id_cliente,
  u.`Codigo Unico`,
  COALESCE(s.ruc, c.ruc) AS ruc,
  c.nombre AS nombre_cliente,
  s.`Segmento FX`,
  s.`Tipo Cuenta`,
  s.logeo,
  s.`Num Logeos`,
  s.`Flg Dig`,
  CASE
    WHEN c.utilidad < 500 AND c.`Codigo Unico` IS NOT NULL THEN 1
    ELSE 0
  END AS flag_fuga
FROM (
```

```

-- Clientes desde SEGMENTOS (ya normalizado)
SELECT DISTINCT
    `Codigo Unico`,
    ruc
FROM `project-sin-477115.refined_data.segmentos`

UNION DISTINCT

-- Clientes desde cli_cambios (renombrando correctamente)
SELECT DISTINCT
    `Codigo Unico` AS codigo_unico,
    ruc
FROM `project-sin-477115.refined_data.cli_cambios`
) u
LEFT JOIN `project-sin-477115.refined_data.segmentos` s
    ON u.`Codigo Unico` = s.`Codigo Unico`
LEFT JOIN temp_cambios_clientes_reciente c
    ON u.`Codigo Unico` = c.`Codigo Unico`
AND c.rn = 1;

```

El objetivo es construir una tabla de dimensiones de clientes que no solo contenga la información básica de identificación y segmentación del cliente, sino que también capture el último estado registrado del cliente, específicamente la última "utilidad" y un indicador de posible fuga.

```

-- DIM_COMUNICACION
CREATE OR REPLACE TABLE `project-sin-477115.ESTRELLA.DIM_COMUNICACION` AS
SELECT
    ROW_NUMBER() OVER (
        ORDER BY `Tipo Cliente`, especificacion, estado, card
    ) AS id_comunicacion,
    `Tipo Cliente`,
    especificacion,
    estado,
    card,
    CASE
        WHEN especificacion = 'SMS' AND cantidad > 10000 THEN 1
        ELSE 0
    END AS flag_presupuesto
FROM (
    SELECT DISTINCT
        `Tipo Cliente`,
        Especificacion,
        estado,
        card,
        cantidad
    FROM `project-sin-477115.refined_data.registro_comunicaciones`
);

```

El propósito es crear una dimensión que catalogue y codifique todas las posibles combinaciones de tipos y estados de comunicación registrados en la tabla registro_comunicaciones. Esta dimensión permite realizar análisis y agrupaciones detalladas sobre cómo, por qué medio y en qué estado se realizan las comunicaciones.

Los siguientes Scripts fueron usados para las consultas y generación de KPIs.

KPIs FINANCIEROS (RESULTADO DEL NEGOCIO)

Utilidad Total

Qué mide: resultado total generado.

```
SELECT SUM(utilidad) AS utilidad_total
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING`;
```

Utilidad Mensual

```
SELECT
  df.anio,
  df.mes_num,
  SUM(f.utilidad) AS utilidad_mensual
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_FECHA` df
  ON f.id_fecha = df.id_fecha
GROUP BY df.anio, df.mes_num;
```

Crecimiento Mes a Mes (MoM %)

```
WITH m AS (
  SELECT
    df.anio,
    df.mes_num,
    SUM(f.utilidad) AS utilidad
  FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
  JOIN `project-sin-477115.ESTRELLA.DIM_FECHA` df
    ON f.id_fecha = df.id_fecha
  GROUP BY df.anio, df.mes_num
)
SELECT
  *,
  SAFE_DIVIDE(
    utilidad - LAG(utilidad) OVER (ORDER BY anio, mes_num),
    LAG(utilidad) OVER (ORDER BY anio, mes_num)
  ) * 100 AS crecimiento_pct
FROM m;
```

KPIs DE VOLUMEN Y EFICIENCIA

Volumen Total Cambiado

```
SELECT SUM(volumen_cambiado) AS volumen_total
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING`;
```

Margen de Utilidad (%)

Qué mide: eficiencia del negocio.

```
SELECT
  SAFE_DIVIDE(SUM(utilidad), SUM(volumen_cambiado)) * 100 AS margen_utilidad
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING`;
```

Volumen vs Utilidad

Detecta operaciones poco rentables.

```
SELECT
  volumen_cambiado,
  utilidad
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING`;
```

KPIs DE CLIENTES

Clientes Activos

```
SELECT COUNT(DISTINCT id_cliente) AS clientes_activos
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING`;
```

Utilidad Promedio por Cliente

```
SELECT AVG(utilidad_cliente) AS utilidad_promedio
FROM (
  SELECT id_cliente, SUM(utilidad) AS utilidad_cliente
  FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING`
  GROUP BY id_cliente
);
```

Top 10 Clientes por Utilidad

```
SELECT
  dc.nombre_cliente,
  SUM(f.utilidad) AS utilidad
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_CLIENTE` dc
  ON f.id_cliente = dc.id_cliente
GROUP BY dc.nombre_cliente
ORDER BY utilidad DESC
LIMIT 10;
```

KPIs COMERCIALES (CANAL / EJECUTIVO)

Utilidad por Canal

```
SELECT
    dca.nombre_canal,
    SUM(f.utilidad) AS utilidad
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_CANAL` dca
    ON f.id_canal = dca.id_canal
GROUP BY dca.nombre_canal;
```

Ranking de Ejecutivos

```
SELECT
    de.nombre_ejecutivo,
    SUM(f.utilidad) AS utilidad
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_EJECUTIVO` de
    ON f.id_ejecutivo = de.id_ejecutivo
GROUP BY de.nombre_ejecutivo
ORDER BY utilidad DESC;
```

Distribución Canal vs Cliente

```
SELECT
    dca.nombre_canal,
    COUNT(DISTINCT f.id_cliente) AS clientes
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_CANAL` dca
    ON f.id_canal = dca.id_canal
GROUP BY dca.nombre_canal;
```

SEGMENTOS

Total Comunicaciones 2025

Cuenta el total de comunicaciones en 2025

```
SELECT
    COUNT(id_comunicacion) AS total_comunicaciones_2025
FROM `project-sin-477115.ESTRELLA.DIM_COMUNICACION` dc
JOIN `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
    ON f.id_comunicacion = dc.id_comunicacion
JOIN `project-sin-477115.ESTRELLA.DIM_FECHA` df
    ON f.id_fecha = df.id_fecha
WHERE df.anio = 2025;
```

Comunicaciones por Mes

Promedio o total mensual


```

SELECT
    df.anio,
    df.mes_num,
    COUNT(f.id_comunicacion) AS comunicaciones_mes
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_FECHA` df
    ON f.id_fecha = df.id_fecha
GROUP BY df.anio, df.mes_num
ORDER BY df.anio, df.mes_num;

```

Comunicaciones por Segmento

Cantidad de comunicaciones por segmento de cliente

```

SELECT
    dc.segmento_fx,
    COUNT(f.id_comunicacion) AS total_comunicaciones
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_CLIENTE` dc
    ON f.id_cliente = dc.id_cliente
GROUP BY dc.segmento_fx
ORDER BY total_comunicaciones DESC;

```

Utilidad por Comunicación

Utilidad generada por segmento a lo largo del tiempo

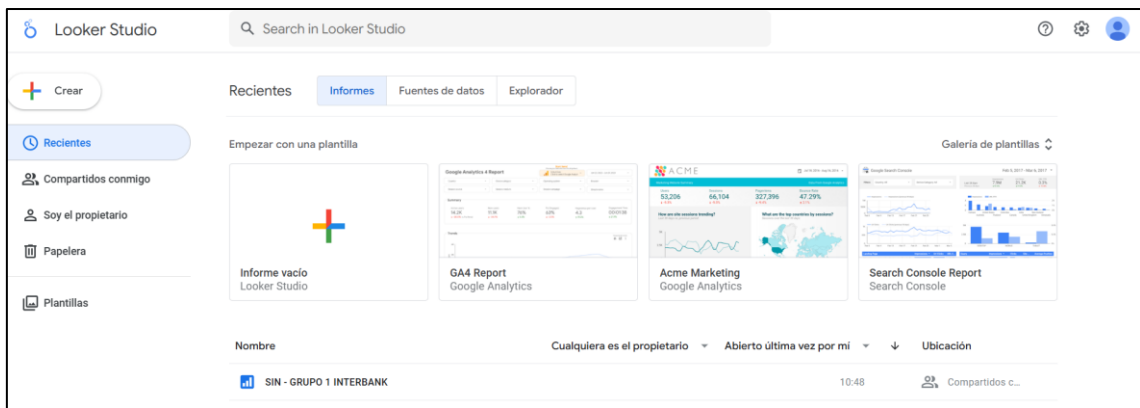
```

SELECT
    df.anio,
    df.mes_num,
    dc.segmento_fx,
    SUM(f.utilidad) AS utilidad_total
FROM `project-sin-477115.ESTRELLA.FACT_UTILIDAD_TRADING` f
JOIN `project-sin-477115.ESTRELLA.DIM_CLIENTE` dc
    ON f.id_cliente = dc.id_cliente
JOIN `project-sin-477115.ESTRELLA.DIM_FECHA` df
    ON f.id_fecha = df.id_fecha
GROUP BY df.anio, df.mes_num, dc.segmento_fx
ORDER BY df.anio, df.mes_num;

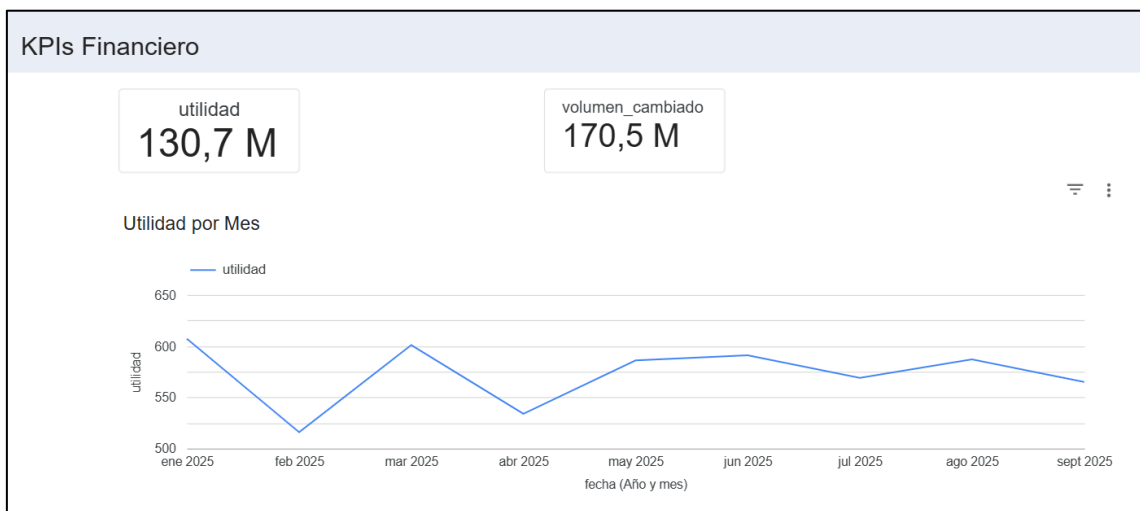
```

VISUALIZACIONES

Las visualizaciones se realizaron en Looker Studio realizando una conexión con BigQuery.

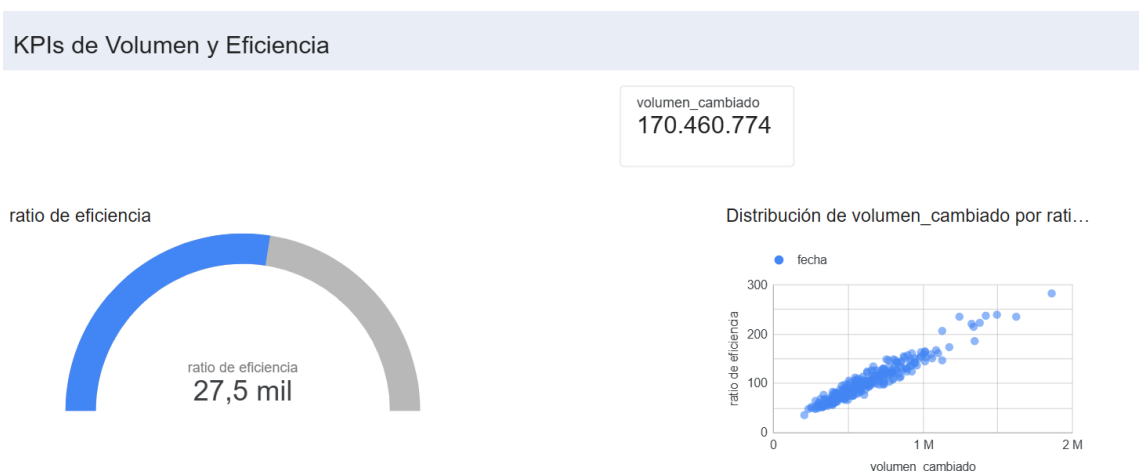


A continuación se muestran algunas visualizaciones:



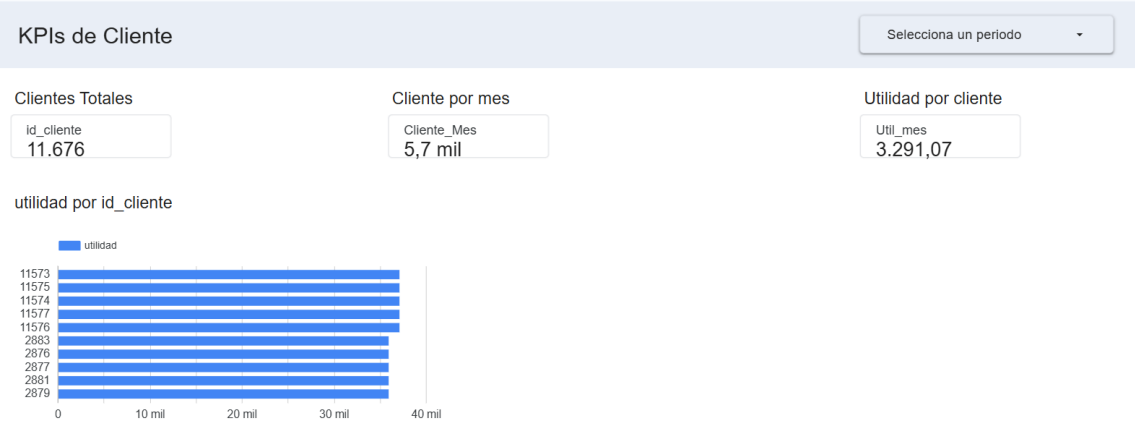
Este panel se enfoca en presentar los KPIs Financieros clave de la organización para el periodo de enero a septiembre de 2025. Se muestra dos métricas acumuladas importantes: la Utilidad total, que asciende a 130.7 millones, y el Volumen Cambiado total, que es de 170.5 millones.

La gráfica inferior es una vista granular de las utilidades por cada mes del año 2025 .

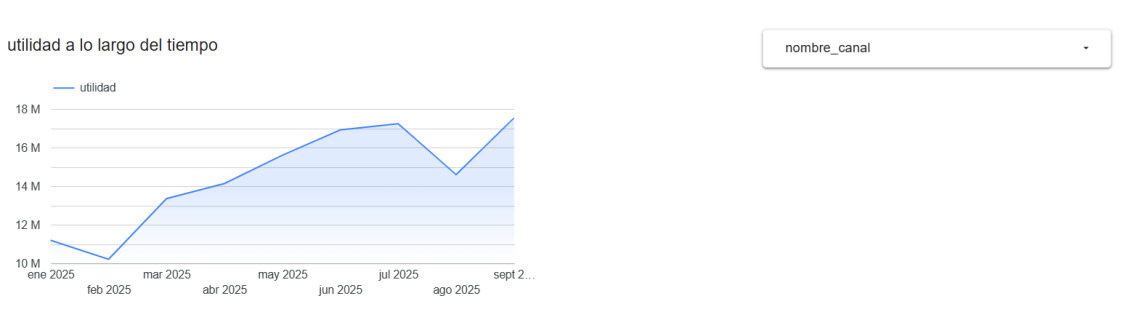


El panel destaca un Volumen Cambiado total de 170.460.774 (que es consistente con el volumen de 170.5M mostrado en el panel financiero). La métrica central es el Ratio de

Eficiencia, que se muestra tanto en un gráfico de velocímetro como en un valor numérico, indicando una eficiencia de 27,5 mil. Finalmente, el dashboard incluye un gráfico de dispersión (scatter plot) titulado "Distribución de volumen_cambiado por ratio" que visualiza la relación entre el volumen_cambiado (en el eje X, hasta 2M) y el ratio de eficiencia (en el eje Y, hasta 300), mostrando una tendencia general donde un mayor volumen cambiado parece correlacionarse con un mayor ratio de eficiencia.

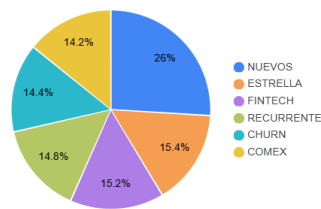


El dashboard indica que hay un total de 11.676 Clientes Totales (Id_cliente). El Cliente por Mes promedio o totalizado es de 5,7 mil (Cliente_Mes), y la Utilidad por cliente promedio es de 3.291,07 (Utilidad por cliente).

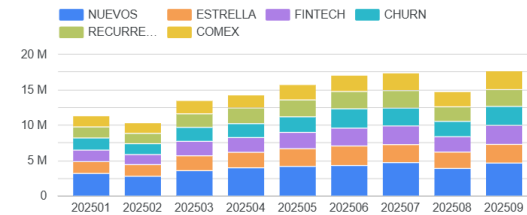


Este gráfico muestra la "Utilidad a lo largo del tiempo" para el período comprendido entre enero y septiembre de 2025. La utilidad tuvo un valor inicial alrededor de 10-11 millones en enero, experimentó una caída en febrero, y luego mostró una tendencia general ascendente hasta alcanzar su punto máximo de casi 18 millones en julio de 2025. Posteriormente, se observa una ligera disminución en agosto, pero cerró el periodo con un nuevo repunte en septiembre de 2025.

segmento_fx por id_cliente



Utilidad por comunicación



Estos gráficos ofrecen una visión integral de quiénes son los clientes (su distribución por segmento) y cómo cada segmento contribuye a la utilidad mensual total de la empresa, destacando la importancia del segmento COMEX y la base de clientes Nuevos.

Gráfico Circular:

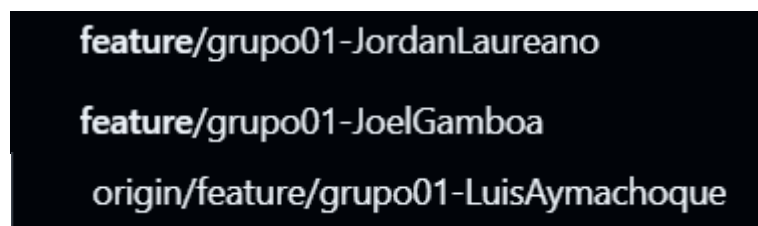
Muestra la distribución porcentual de los clientes (id_cliente) a través de diferentes segmentos definidos. Los segmentos con mayor proporción de clientes son Nuevos (26%), seguidos por Recurrente (14.6%), Churn (14.4%), y COMEX (14.2%). Esto indica que la base de clientes está dominada por clientes nuevos, con una distribución relativamente pareja entre los otros segmentos.

Gráfico de Barras:

Muestra la Utilidad Total (eje Y, hasta 20M) distribuida por segmento (Segmento FX) y a lo largo del tiempo (eje X, por mes, desde enero hasta septiembre de 2025). La utilidad total mensual se mantuvo estable entre 10M y 17.5M a lo largo del periodo. Se observa un crecimiento constante en la utilidad total desde enero (202501) hasta julio (202507), con una ligera disminución en agosto (202508), y un nuevo máximo en septiembre (202509). El segmento COMEX (amarillo) parece ser el mayor contribuyente a la utilidad total en la mayoría de los meses, seguido de cerca por el segmento NUEVOS (azul). La utilidad generada por el segmento FINTECH (morado) muestra una tendencia visiblemente creciente a partir de marzo de 2025 (202503).

REPOSITORIO GITHUB

SI807_Cloud_BI_2025 es el repositorio del curso en el cual se creó 3 ramas:



Cada rama se relaciona a cada integrante del grupo para poder realizar las actividades que les correspondía a cada uno con el fin de completar el proyecto.

En este caso la Rama Líder es de Jordan Laureano el cuál se encargó de realizar la revisión y aceptación de los pull request que hacían los miembros del equipo: Joel Gamboa y Luis Aymachoque.

JordanLau21 authored 2 weeks ago

Delete grupo01_interbank/TT	SI807-FIS-UNI/SI807_Cloud_BI_ on Dec 1	RADA.ipynb	Verified	e9fac51		
Feature/grupo01 joel gamboa #95			Verified	30afd01		
Create SCRIPTS_PYSPARK.py			Verified	0463d57		
Update LIMPIEZA_MEJORAD	Añadido de configuraciones de IAM y GCS en README evidencias		Verified	963c502		
Create LIMPIEZA_MEJORAD	feature/grupo01-Jor... -- feature/grupo01-Joe...		Verified	eff2c8a		

You opened this pull request

Feature/grupo01 joel gamboa (#95)

JordanLau21 authored 2 weeks ago

Joel-Gamboa-17 commented 2 weeks ago Member

Añadido de configuraciones de IAM y GCS en README evidencias

Joel-Gamboa-17 added 4 commits 2 weeks ago

- Create README.md Verified [82f3f0a](#)
- Add files via upload Verified [5216041](#)
- Rename Diagrama FLUJO arquitectura NUBE.png to Diagrama _arquitectura_ Verified [c3746e6](#)
- Configuracion IAM y GCS Verified [e4200f1](#)

JordanLau21 merged commit [eff2c8a](#) into [feature/grupo01-JordanLaureano](#) 2 weeks ago Revert

Pull request successfully merged and closed Delete branch

You're all set — the [feature/grupo01-JoelGamboa](#) branch can be safely deleted.

LJairo26 and others added 5 commits last month

- Update README.md b8f5867
- Merge branch 'feature/grupo01-JordanLaureano' into feature/grupo01-Lu... acc4902
- Update README.md 72e86f7
- Update README.md Verified [6d45065](#)
- Merge branch 'feature/grupo01-JordanLaureano' into origin/feature/gru... Verified [9d6b494](#)

JordanLau21 merged commit [994bee0](#) into [feature/grupo01-JordanLaureano](#) last month Revert

Pull request successfully merged and closed

You're all set — the branch has been merged.

Los merged fueron revisados y aceptados por el líder para integrar el proyecto en su rama Líder.

DOCS	Update README.md
ETL	Add files via upload
Evidencias	Update README.md
Diagrama _arquitectura_nube.png	Rename Diagrama FLUJO arquitectura NUBE.png to Diagrama _arquitectura...

La estructura quedaría como se muestra en la imagen en la rama del Líder.